

计算机组成原理

王鸿鹏

wanghp@hit.edu.cn

计算机科学与技术学院

第九章 并行处理器

- 并行处理器与并行程序
- GPU与多处理器网络拓扑
- 多处理器测试基准和性能模型

指令级并行(ILP)

- 流水线技术挖掘了指令间潜在的并行性，这种并行性被称为指令级并行
- 提高指令集并行度主要有两种方法
 - 增加流水线级数
 - 每个阶段更少的工作，时钟周期可以变短
 - 多发射
 - 增加流水线内部的功能部件数量，一个时钟周期内发射多条指令
 - CPI可以小于 1，使用IPC（CPI的倒数）来衡量
 - 例如，4GHz 发射宽度为4的多发射处理器：16 BIPS (Billion Instructions Per Second), 最高CPI = 0.25, 最高IPC = 4
 - 多发射技术会有一些限制，比如哪些指令可以同时执行、如果发生冒险如何处理等

静态多发射的RISC-V处理器

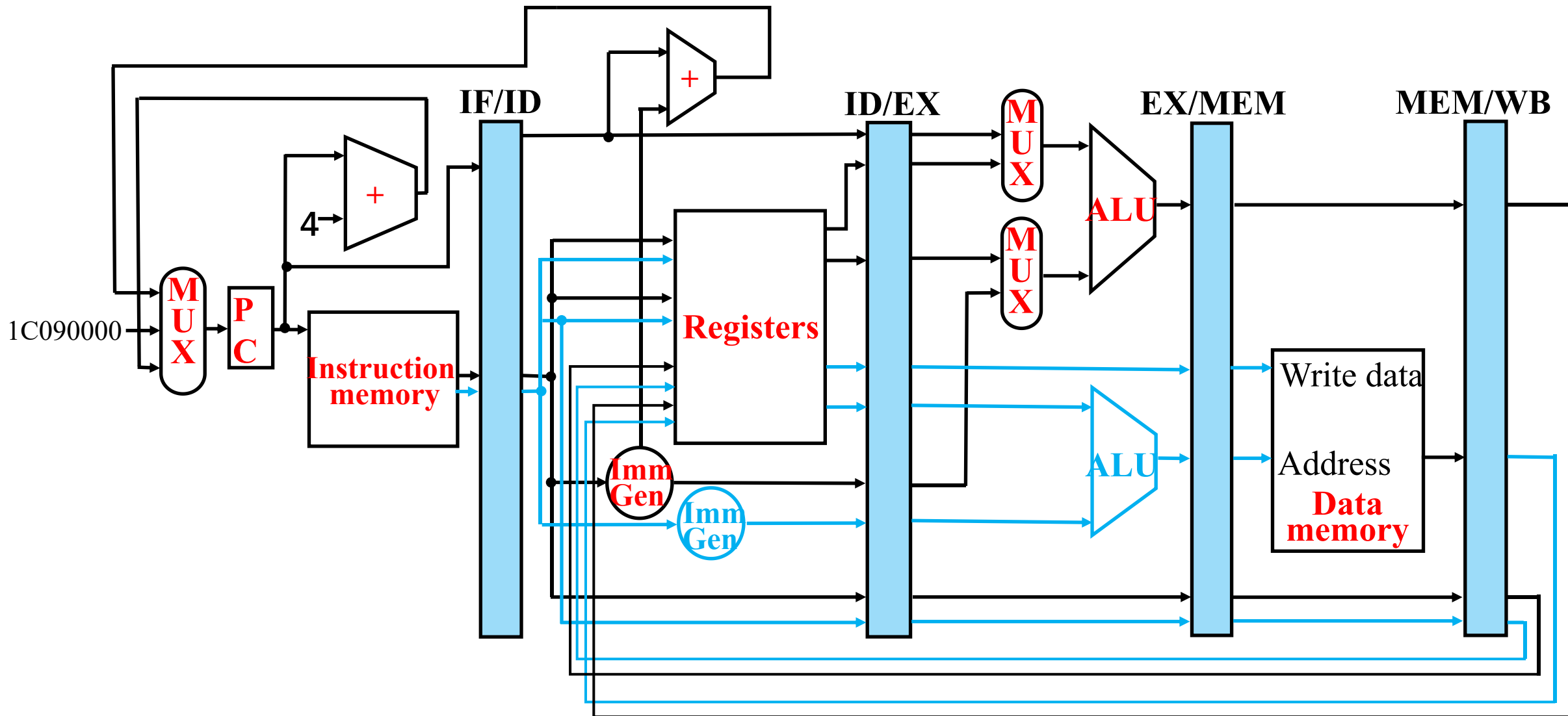
- 假设是双发射处理器（单个周期内发射两条指令）
 - ALU指令或分支指令放在前面
 - load指令或store指令放在后面
 - 如果一条指令无法发射，替换成nop指令
 - 指令地址需要64位边界对齐

Address	Instruction type	Pipeline Stages						
n	ALU/branch	IF	ID	EX	MEM	WB		
n + 4	Load/store	IF	ID	EX	MEM	WB		
n + 8	ALU/branch		IF	ID	EX	MEM	WB	
n + 12	Load/store		IF	ID	EX	MEM	WB	
n + 16	ALU/branch			IF	ID	EX	MEM	WB
n + 20	Load/store			IF	ID	EX	MEM	WB

需要增加的硬件资源

- 冒险检测和流水线停顿逻辑
 - 假设在这个例子中，编译器只负责单个指令包中检测所有类型的冒险；硬件来检测两个指令包之间的数据冒险
- 寄存器堆读写口
 - 两条指令需要同时取指和译码
- 加法器
 - ALU部件只负责ALU指令的执行，还需要加法器来进行访存地址的计算

静态双发射数据通路



并行处理器与并行程序简介

- 目标：连接多台计算机以获得更高的性能
 - 多处理器
 - 可扩展性、可用性和能效
- 任务级（进程级）并行
 - 独立作业的高吞吐量
- 并行处理程序
 - 单个程序在多个处理器上运行
- 多核微处理器（multicore microprocessor）
 - 多处理器芯片（核心）

硬件和软件

- 硬件
 - 串行：例如，奔腾4
 - 并行：例如，四核至强e5345
- 软件
 - 顺序：例如，矩阵乘法
 - 并发：例如，操作系统
- 顺序/并发的软件可以在串行/并行硬件上运行
 - 挑战：有效利用并行硬件

并行编程

- 并行性的问题在于软件
- 需要获得显著的性能提升
 - 否则，就使用更快的单处理器，因为更容易实现
- 难点
 - 任务划分和调度
 - 负载均衡
 - 通信开销

加速比的挑战

- 顺序部分会限制速度提升(Amdahl定律)
- 假设一个程序在一台计算机上运行需要100秒，其中80秒用于乘法运算。如果要把该程序的运行速度提高5倍，乘法运算的速度应该改进多少？

改进后的时间=改进部分的执行时间/改进量 + 未改进部分的时间

$$100/5 = 80/x + (100-80) \longrightarrow x \text{ 无穷大}$$

- 边际收益递减定律

Amdahl定律

- 例子：假如你希望在100个处理器上实现90倍的加速，那么原始计算中的百分之多少是可以顺序执行的？
 - $T_{\text{new}} = T_{\text{parallelizable}}/100 + T_{\text{sequential}}$
 - 加速比 = $1 / [(1 - F_{\text{parallelizable}}) + (F_{\text{parallelizable}}/100)] = 90$
 - 结论: $F_{\text{parallelizable}} = 0.999$
- 顺序执行的程序最多占0.1%

缩放（Scaling）示例

- 要计算两个加法：一个加法是10个标量求和，另一个加法是一对 10×10 的二维数组求矩阵和（假设：矩阵求和可以并行化，负载可以在处理器之间平衡）。分别计算10个处理器和100个处理器时的加速比。

单个处理器：Time = $(10 + 100) \times t_{\text{add}} = 110 \times t_{\text{add}}$

10个处理器

$$\text{Time} = 10 \times t_{\text{add}} + 100/10 \times t_{\text{add}} = 20 \times t_{\text{add}}$$

$$\text{Speedup} = 110/20 = 5.5 \text{ (潜在加速比为55\%)}$$

100个处理器

$$\text{Time} = 10 \times t_{\text{add}} + 100/100 \times t_{\text{add}} = 11 \times t_{\text{add}}$$

$$\text{Speedup} = 110/11 = 10 \text{ (潜在加速比为10\%)}$$

缩放示例—续

- 如果矩阵维数变为 100×100 呢? (假设负载平衡)
- 单个处理器: $\text{Time} = (10 + 10000) \times t_{\text{add}}$
- 10个处理器
 - $\text{Time} = 10 \times t_{\text{add}} + 10000/10 \times t_{\text{add}} = 1010 \times t_{\text{add}}$
 - $\text{Speedup} = 10010/1010 = 9.9$ (潜在加速比为99%)
- 100个处理器
 - $\text{Time} = 10 \times t_{\text{add}} + 10000/100 \times t_{\text{add}} = 110 \times t_{\text{add}}$
 - $\text{Speedup} = 10010/110 = 91$ (潜在加速比为91%)

强比例缩放和弱比例缩放（Scaling）

- 强比例缩放：问题规模不变
 - 如上述例子
- 弱比例缩放：问题大小与处理器数量成正比
 - 10个处理器， 10×10 的矩阵
 - $\text{Time} = 20 \times t_{\text{add}}$
 - 100个处理器， 32×32 的矩阵
 - $\text{Time} \approx 10 \times t_{\text{add}} + 1000/100 \times t_{\text{add}} = 20 \times t_{\text{add}}$
 - 性能不变

指令流和数据流

- 基于指令流数量和数据流数量的硬件分类实例

		数据流	
		单数据流	多数据流
指令流	单指令流	SISD : Intel Pentium 4	SIMD : x86的SSE指令
	多指令流	MISD :理论模型	MIMD :Intel Xeon e5345

- **SPMD**: 单程序多数据流
 - 一种MIMD计算机上的并程序序
 - 用于不同处理器的条件代码

向量机

- SIMD——向量体系结构
- 数据流从向量寄存器到单元
 - 从内存收集数据到寄存器
 - 使用流水化的执行单元操作寄存器数据，然后写回到内存
- 例子： RISC-V的向量扩展
 - v0-v31： 32个向量寄存器，每个寄存器包含64个64位的数据元
 - 向量指令示例
 - fld.v, fsd.v: 向量加载与向量存储
 - fadd.d.v: 将两个双精度向量相加
 - fadd.d.vs: 将标量寄存器中的内容加到向量寄存器中的每个数据元上
- 大幅降低指令获取带宽

例子: DAXPY ($Y = a \times X + Y$)

传统RISC-V代码:

```
fld  f0,a(x3)    // load scalar a
addi  x5,x19,512  // end of array X
loop: fld  f1,0(x19) // load x[i]
      fmul.d f1,f1,f0 // a * x[i]
      fld  f2,0(x20) // load y[i]
      fadd.d f2,f2,f1 // a * x[i] + y[i]
      fsd  f2,0(x20) // store y[i]
      addi  x19,x19,8 // increment index to x
      addi  x20,x20,8 // increment index to y
      bltu  x19,x5,loop // repeat if not done
```

向量RISC-V代码

```
fld  f0,a(x3)    // load scalar a
fld.v  v0,0(x19) // load vector x
fmul.d.vs v0,v0,f0 // vector-scalar multiply
fld.v  v1,0(x20) // load vector y
fadd.d.v v1,v1,v0 // vector-vector add
fsd.v  v1,0(x20) // store vector y
```

向量与标量

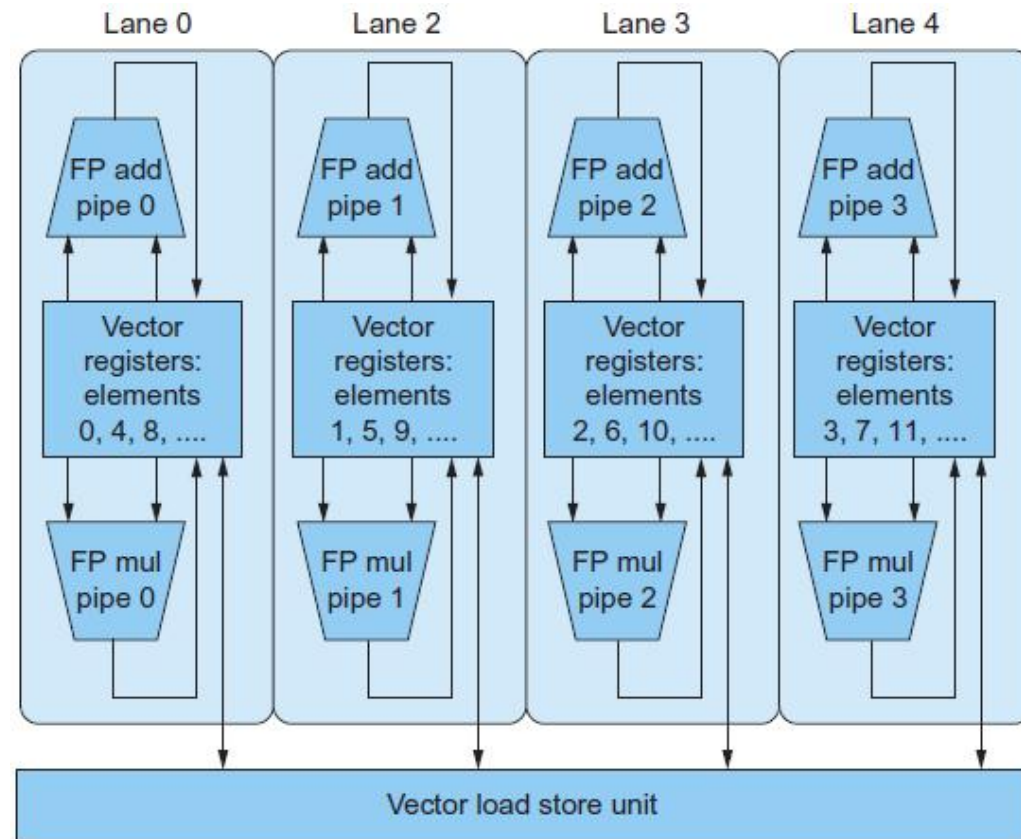
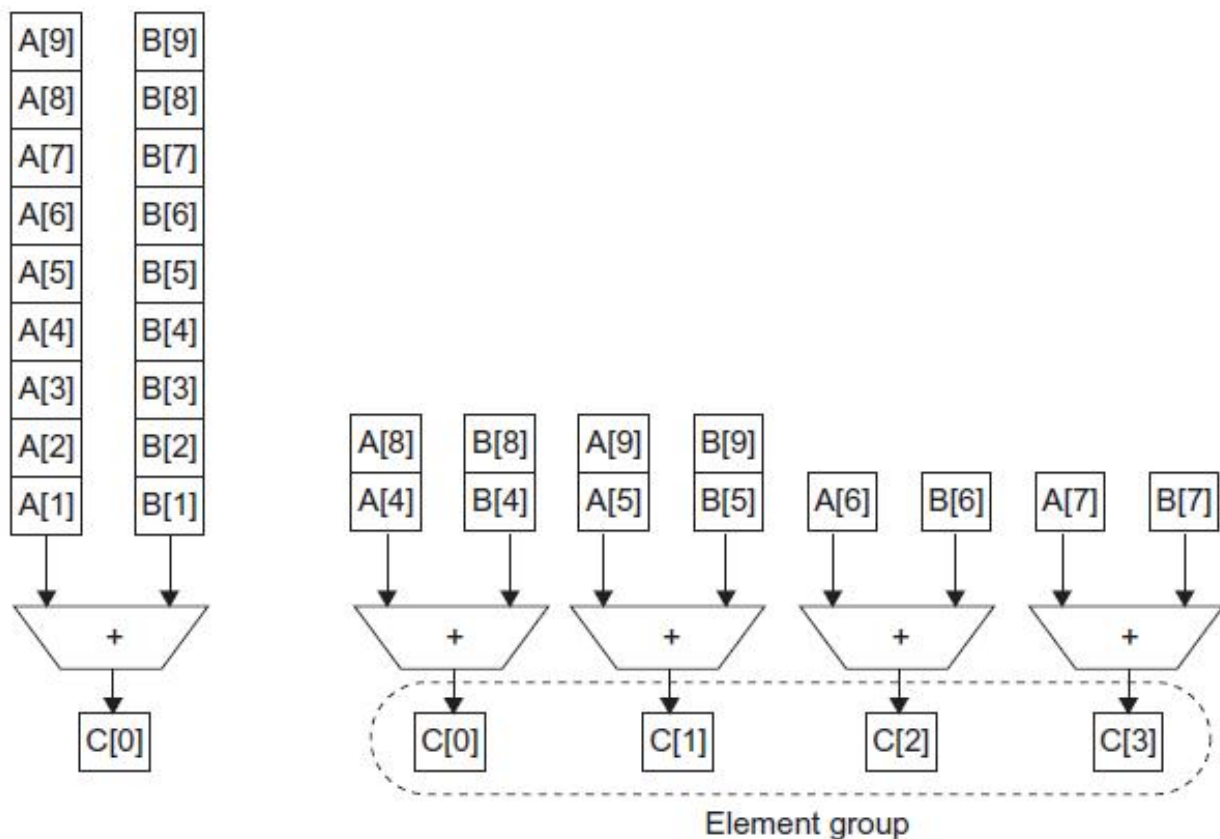
- 向量体系结构和编译器
 - 简化数据并行编程
 - 确认向量中的每个结果都是独立的，没有循环依赖
 - 减少了硬件中的检查（数据冒险）
 - 常规访问模式受益于交错式和突发式内存
 - 通过避免循环来避免控制冒险
- 比临时性的媒体扩展（如MMX、SSE）更普遍
 - 与编译器技术更匹配

SIMD (Single Instruction Multiple Data Stream)

- 对数据的向量进行元操作
 - 例如，X86中的MMX和SSE指令
 - 128位宽寄存器中的多个数据元素
- 所有处理器在同一时间执行相同的指令
 - 每个处理器有不同的数据地址
- 简化了同步操作
- 减少了指令控制硬件
- 对高度数据并行的应用效果最好

向量与多媒体扩展

- 向量指令有一个可变的向量宽度，多媒体扩展有一个固定的宽度
- 向量指令支持步长访问，多媒体扩展不支持
- 向量单元可以是流水线和功能单元阵列的组合。



多线程

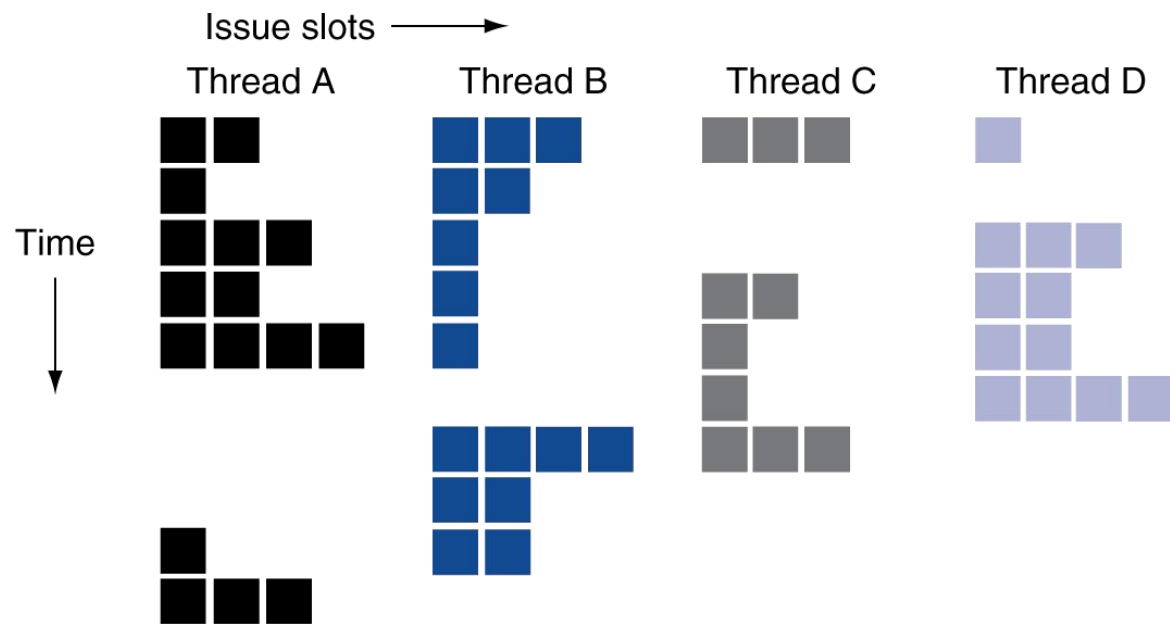
- 并行进行多线程的执行
 - 需要复制寄存器，例如，PC等。
 - 在线程之间快速切换
- 细粒度多线程
 - 在每个指令执行后切换线程
 - 交叉执行指令
 - 如果一个线程停顿时，其他线程将被执行（隐藏短时停顿）
- 粗粒度多线程
 - 只在高开销的停顿上切换线程（如末级cache失效）
 - 简化了硬件，但不能隐藏短时停顿（例如，数据冒险）

同时多线程(SMT)

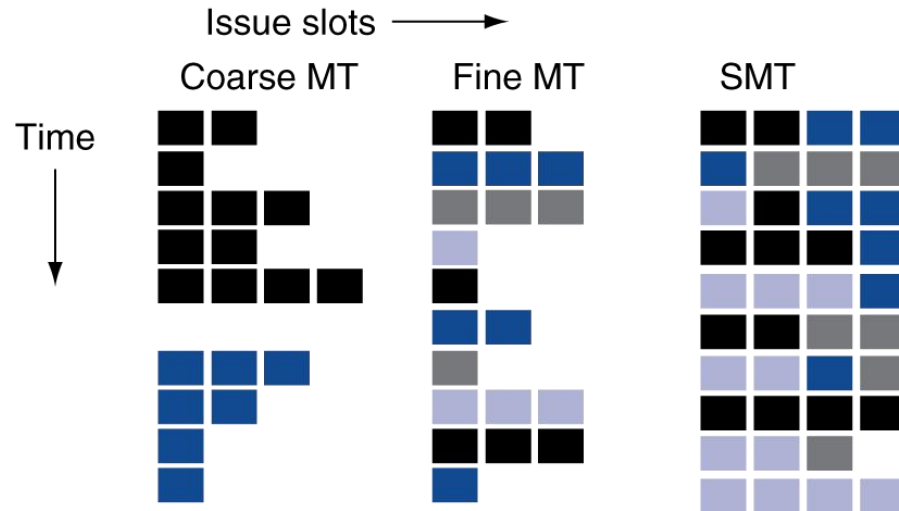
- Simultaneous MultiThreading
- 在多发射、动态调度处理器中
 - 调度来自多线程的指令
 - 当功能单元可用时，独立线程的指令执行
 - 在线程中，通过动态调度和寄存器重命名处理依赖关系
- 例子：Intel Pentium-4 HT
 - 两个线程：复制寄存器、共享功能单元和缓存

多线程示例

- 不支持多线程的超标量处理器



- 支持粗粒度多线程
 - 仅在高开销停顿上切换线程
- 支持细粒度多线程
 - 每条指令执行后进行线程切换
- 支持SMT
 - 资源分配由硬件完成，充分利用

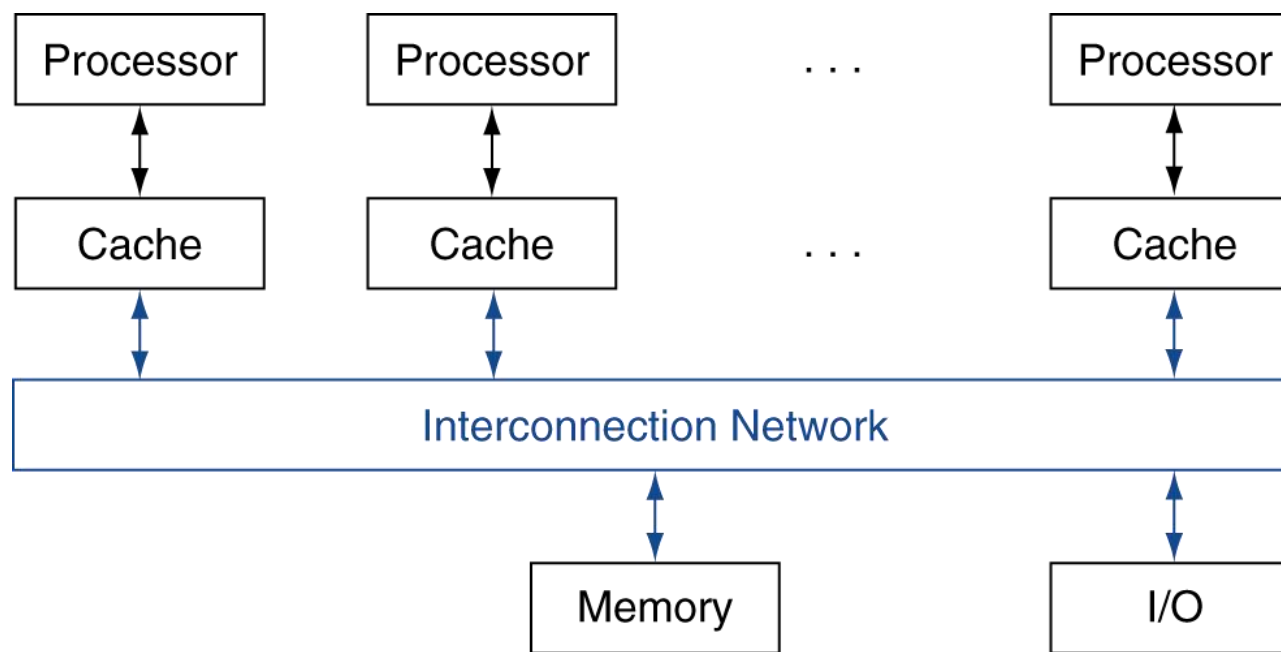


多线程的未来

- 它能生存吗？以何种形式？
- 功耗考虑 $\Rightarrow$ 简化的微体系结构
 - 简化的多线程形式
- 容忍高速缓存缺失的延迟
 - 线程切换可能是最有效的
- 多个简单内核可能更有效地分享资源

共享内存

- SMP: 共享内存多处理器 (Shared Memory Processor)
 - 硬件为所有处理器提供单一物理地址空间
 - 使用锁来同步共享变量
 - 内存访问时间
 - UMA (统一)、NUMA (非统一)



例子: Sum Reduction

- 在64个处理器的UMA上计算64,000个数字的总和
 - 每个处理器ID编号: $0 \leq P_n \leq 63$
 - 每个处理器划分1000个数字
 - 在每个处理器上进行初始求和

```
sum[Pn] = 0;  
for (i = 1000*Pn; i < 1000*(Pn+1); i += 1)  
    sum[Pn] += A[i];
```
- 现在需要把这些部分和加起来
 - 减少: 分治法
 - 先划分一半的处理器加法操作再划分到四分之一 ...
 - 需要在划分步骤之间进行同步

例子: Sum Reduction

```
half = 64; //假设有64核
```

```
do
```

```
    synch(); //等待部分和计算完成
```

```
    if (half%2 != 0 && Pn == 0)
```

```
        sum[0] += sum[half-1];
```

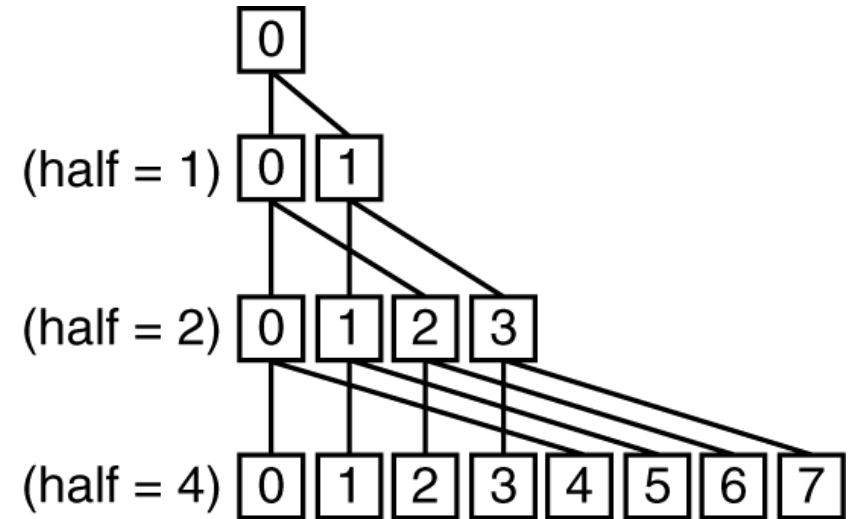
```
    /*当half是奇数时，需要有条件地求和;
```

```
       处理器0得到missing元素*/
```

```
    half = half/2; /* dividing line on who sums */
```

```
    if (Pn < half) sum[Pn] += sum[Pn+half];
```

```
while (half > 1);
```



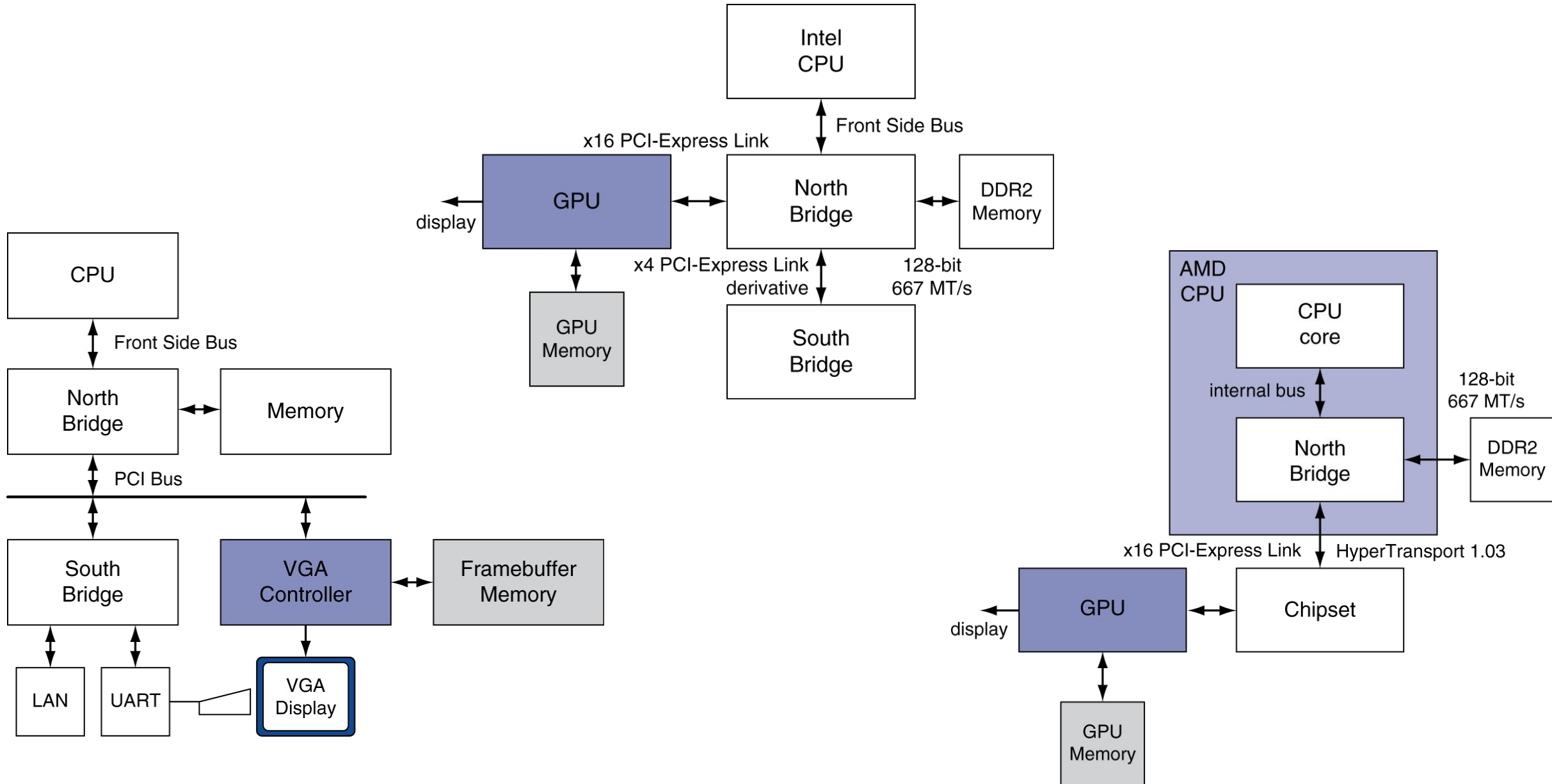
第九章 并行处理器

- 并行处理器与并行程序
- **GPU与多处理器网络拓扑**
- 多处理器测试基准和性能模型

GPU历史发展

- 早期的视频卡
 - 带有视频输出地址生成的缓冲帧存储器
- 3D图形处理
 - 最初是终端计算机（如SGI）。
 - 摩尔定律 降低成本，提高密度
 - 用于PC和游戏机的3D图形卡
- 图形处理单元
 - 面向3D图形任务的处理器
 - 顶点/像素处理、着色、纹理映射、光栅化

Graphics in the System

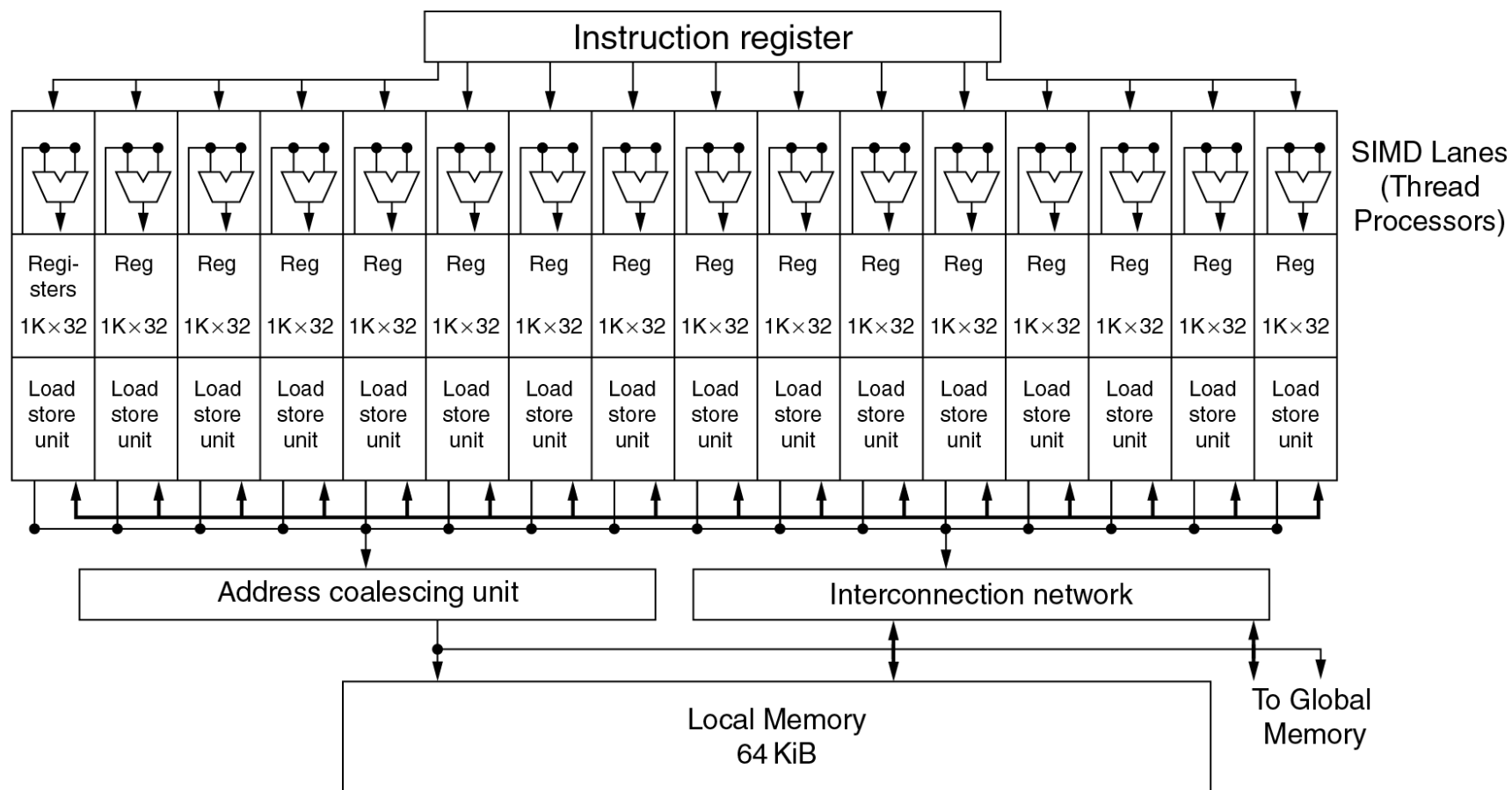


GPU 架构

- 处理数据的过程是高度并行的
 - GPUs 是多线程的
 - 使用线程切换来隐藏内存延迟
 - 这对多级缓存的依赖性较低
 - 显存足够大且高带宽
- 趋向于通用的GPU
 - 异构的CPU/GPU系统
 - CPU适用于顺序代码，GPU适用于并行代码
- 编程语言/APIs
 - DirectX, OpenGL
 - C for Graphics (Cg), High Level Shader Language (HLSL)
 - Compute Unified Device Architecture (CUDA)

Example: NVIDIA Fermi

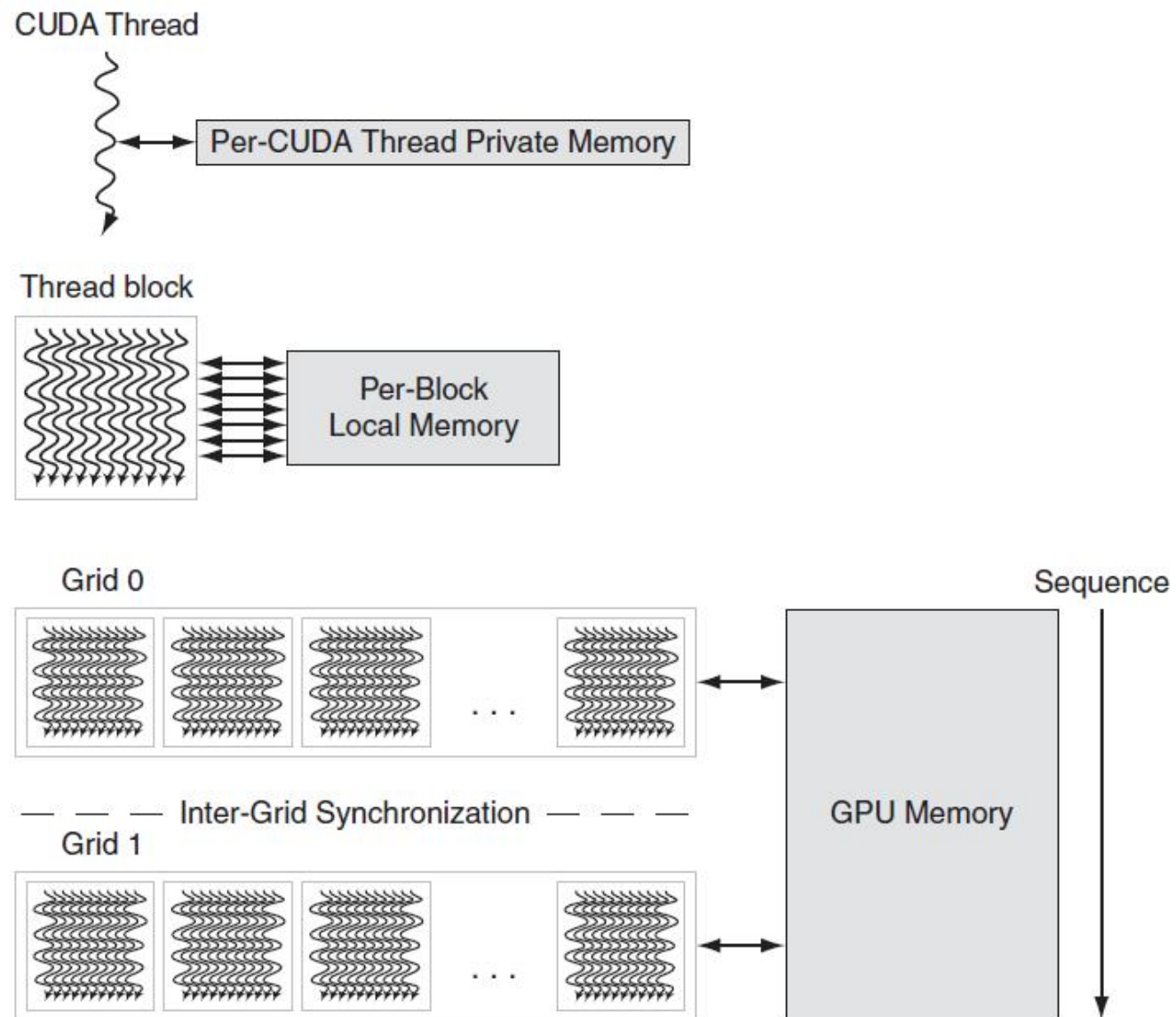
- 多个SIMD处理器:



例子: NVIDIA Fermi

- SIMD处理器: 16条SIMD通道
- SIMD指令
 - 在32位宽的线程上操作运行
 - 在2个周期内动态地安排在16位宽的处理器的上
- 32K x 32位寄存器分布在lanes上
 - 每个线程上下文有64个寄存器

GPU 显存结构

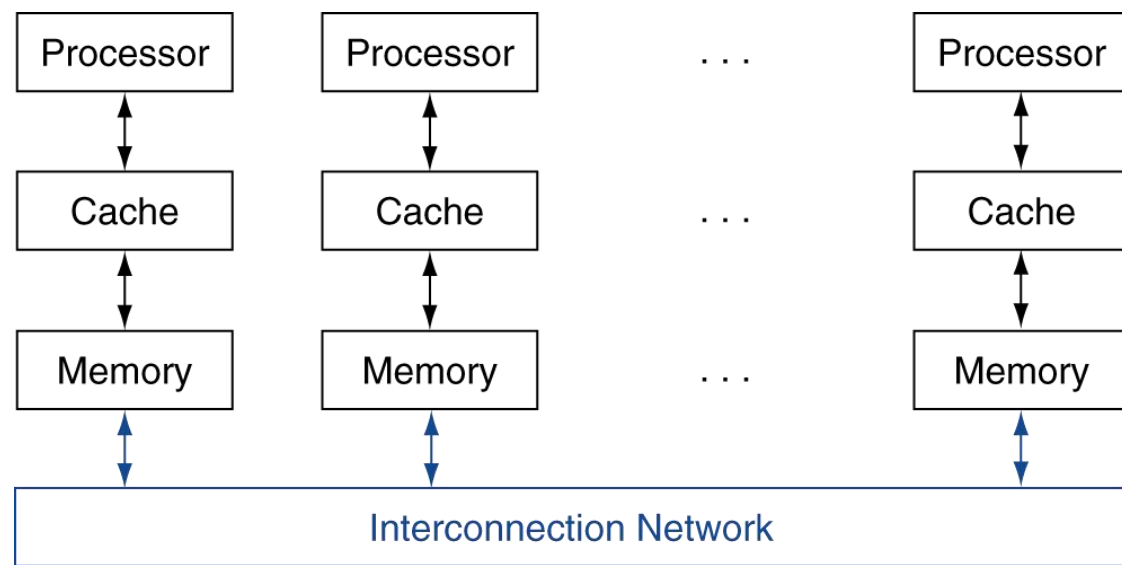


带有SIMD扩展的多核处理器与GPU对比

Feature	Multicore with SIMD	GPU
SIMD processors	4 to 8	8 to 16
SIMD lanes/processor	2 to 4	8 to 16
Multithreading hardware support for SIMD threads	2 to 4	16 to 32
Typical ratio of single precision to double-precision performance	2:1	2:1
Largest cache size	8 MB	0.75 MB
Size of memory address	64-bit	64-bit
Size of main memory	8 GB to 256 GB	4 GB to 6 GB
Memory protection at level of page	Yes	Yes
Demand paging	Yes	No
Integrated scalar processor/SIMD processor	Yes	No
Cache coherent	Yes	No

信息传递

- 每个处理器都有私有的物理地址空间
- 硬件在处理器之间发送/接收信息



松散耦合的集群

- 独立计算机的网络
 - 每个人都有私人内存空间和操作系统
 - 使用I/O系统进行连接
 - 例如，以太网/交换机、互联网
- 适用于具有独立任务的应用
 - 网络服务器，数据库，仿真等...
- 高可用性、可扩展、可负担
- 问题
 - 管理成本（故倾向于虚拟机）
 - 互联带宽低
 - 例如，SMP上的处理器/内存带宽。

Sum Reduction (Again)

- 在64颗处理器上计算64,000个数字的总和
- 首先将1000个数字分配给每个处理器
 - 进行部分求和
sum = 0;
for (i = 0; i < 1000; i += 1)
sum += AN[i];
- 划分任务
 - 一半的处理器发送，另一半接收并添加
 - 四分之一发送，四分之一接收和添加，...

Sum Reduction (Again)

- 假设已经实现send()和receive()

```
limit = 64; half = 64; /* 64位处理器*/
```

```
do
```

```
half = (half+1)/2; /*发送与接收的分界线 */
```

```
if (Pn >= half && Pn < limit)
```

```
    send(Pn - half, sum);
```

```
if (Pn < (limit/2))
```

```
    sum += receive();
```

```
limit = half; /* upper limit of senders */
```

```
while (half > 1); /* exit with final sum */
```

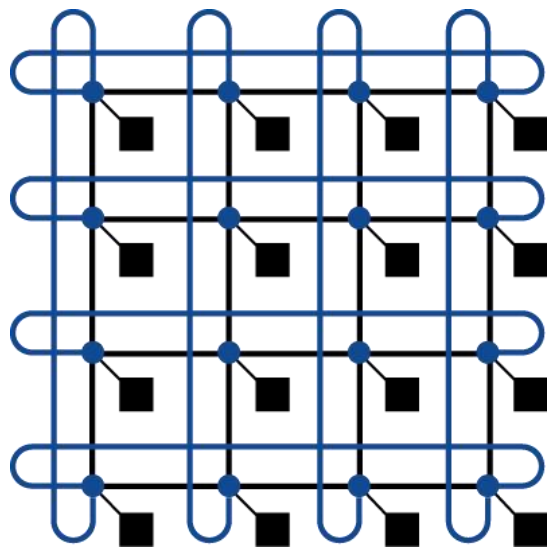
- 发送/接收也提供同步功能
- 假设发送/接收的时间与增加的时间相似

互联网络

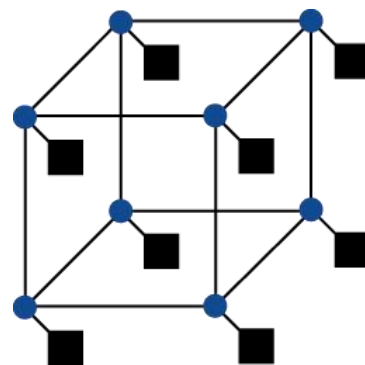
- 网络的拓扑结构
 - 处理器、交换机和链路的安排



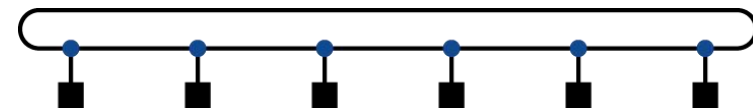
Bus



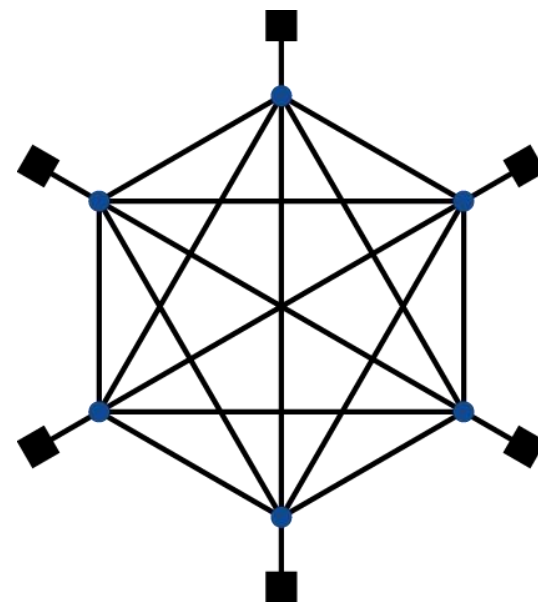
2D Mesh



N-cube ($N = 3$)



Ring



Fully connected

第九章 并行处理器

- 并行处理器与并行程序
- GPU与多处理器网络拓扑
- 多处理器测试基准和性能模型

并行的测试基准

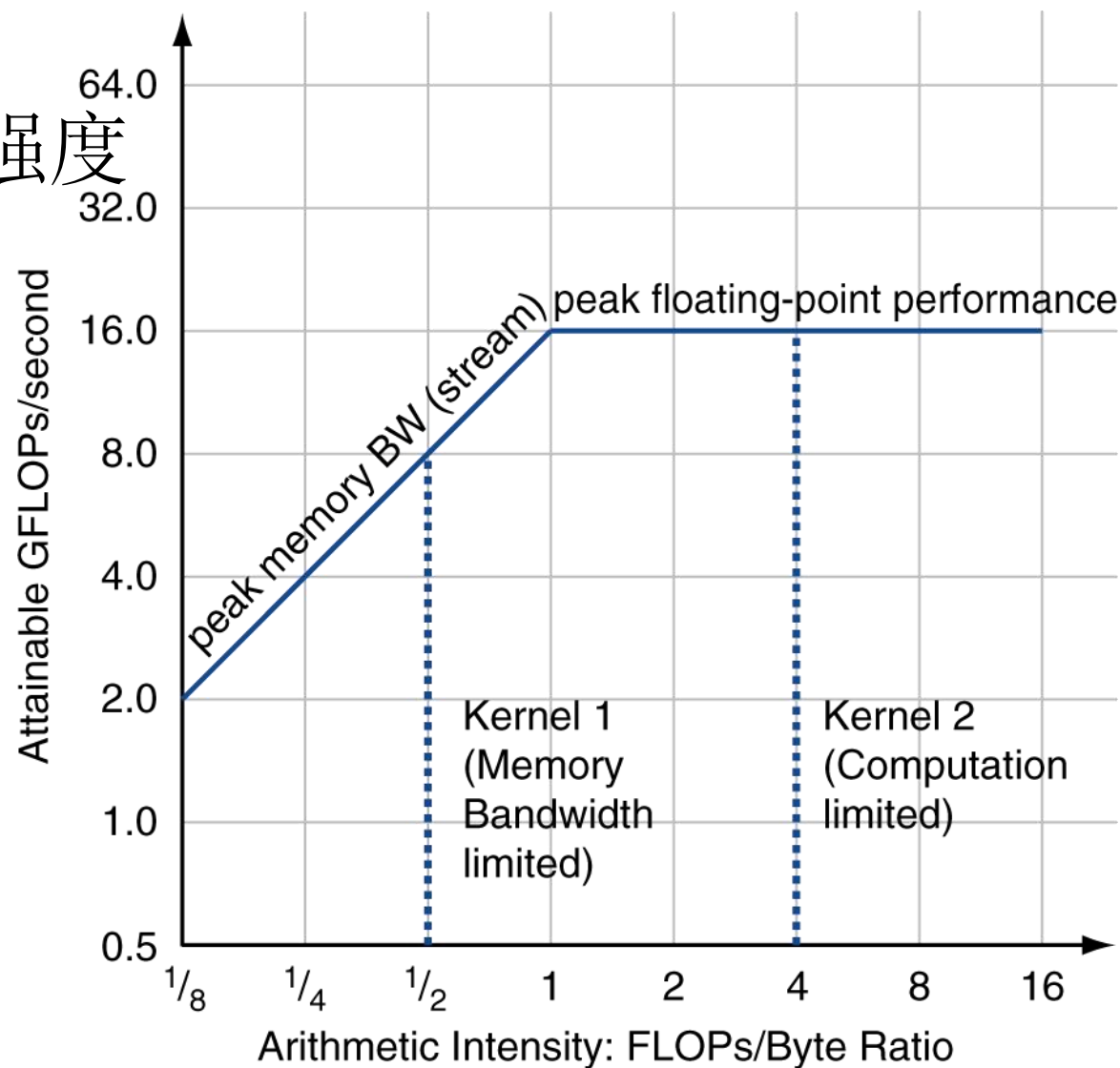
- Linpack: 一组线性代数例程
- SPECrte: 并行运行SPEC CPU程序
 - 任务级并行
- SPLASH: 使用SPLASH2(Stanford Parallel Applications for Shared Memory)作为测试
 - 它由核心程序和应用程序组成，需要强比例缩放
- NAS (NASA Advanced Supercomputing) suite
 - 由5个来源于流体动力学的核心程序组成
- PARSEC (Princeton Application Repository for Shared Memory Computers) suite
 - 由Pthreads和OpenMP的多线程程序组成

性能模型

- 假设感兴趣的性能指标是GFLOPs
- 使用Berkeley Design Patterns的核心程序集进行测量
- 算术强度
 - 一个程序中浮点操作数量与访问内存字节数量的比值
- 对于一个给定的计算机，决定了
 - 峰值浮点性能
 - 峰值内存性能

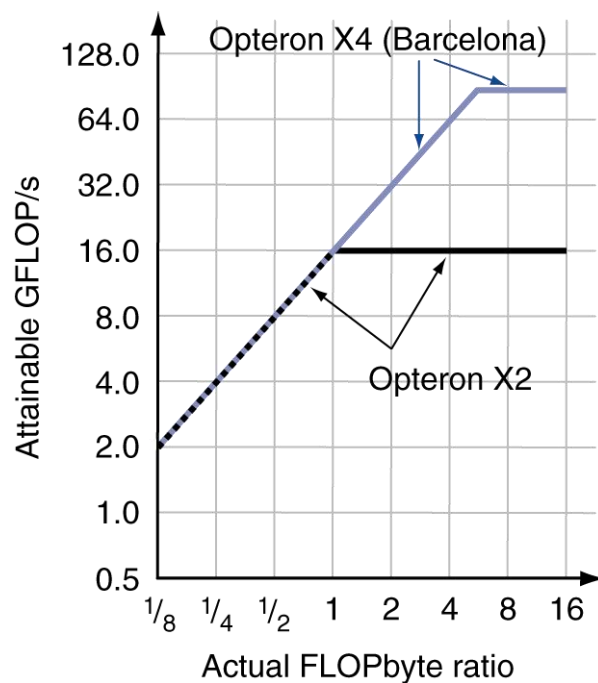
Roofline 模型

可达到的 GFLOPs/sec
= Max (峰值存储带宽 × 算术强度
, 峰值浮点性能)



两代Opteron比较

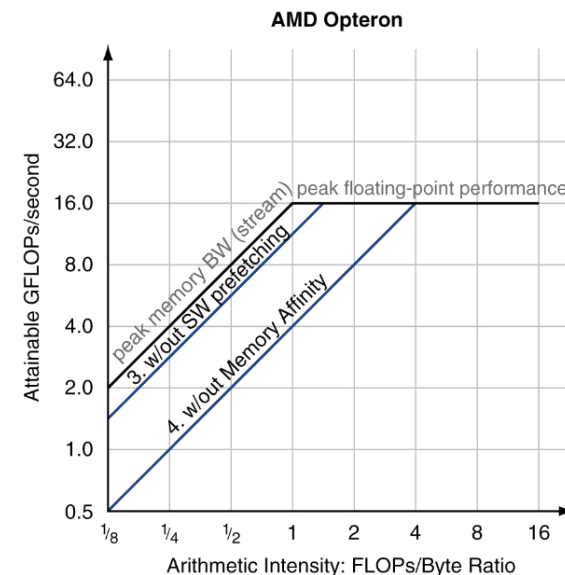
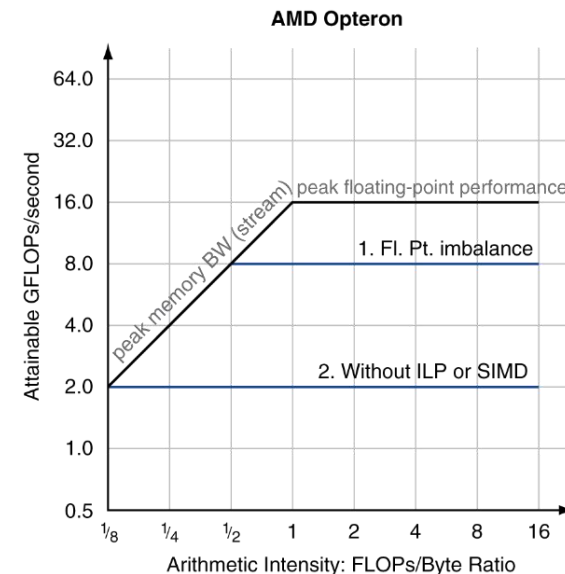
- Example: Opteron X2 vs. Opteron X4
 - 两核 vs. 四核, 四核的每个核的峰值浮点性能提高到原来两倍, 时钟速率 2.2GHz vs. 2.3GHz
 - 存储系统相同



- 为了看到X4比X2更高的性能
 - 核心程序算术强度需要大于1
 - 或者核心程序的工作集必须适合X4的cache

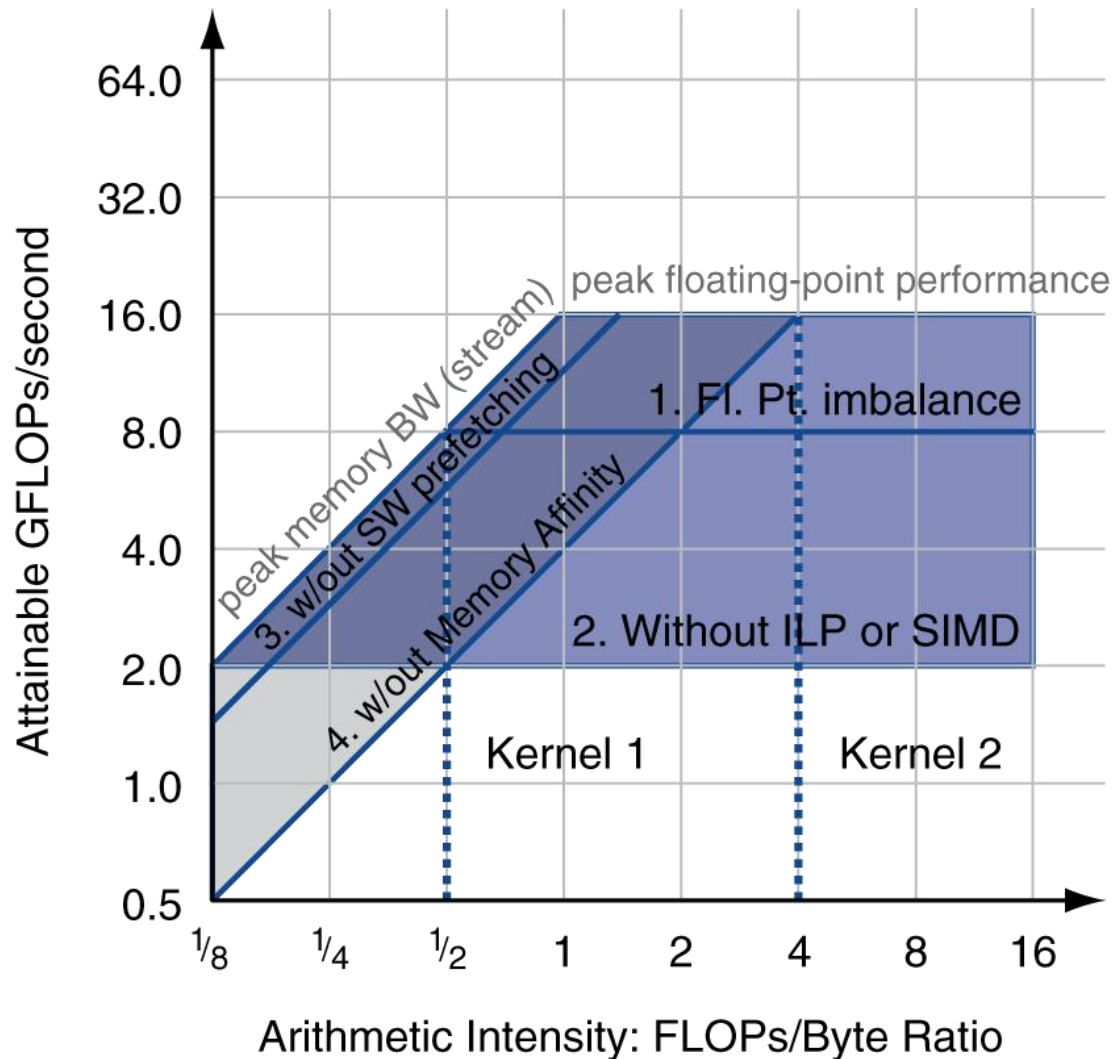
优化性能

- 优化浮点运算性能
 - 均衡加法核乘法
 - 提高指令级并行性并应用SIMD
- 优化存储使用
 - 软件预取
 - 避免访存停顿
 - 内存关联
 - 避免非本地的数据访问



优化性能

- 核心程序的算术强度决定了优化手段的选择



- 算术强度不是一直固定的
 - 可能随问题规模增长
 - 缓存减少访存次数
 - 提高算术强度

基准测试结果

Kernel	Units	Core i7-960	GTX 280	GTX 280/ i7-960
SGEMM	GFLOP/sec	94	364	3.9
MC	Billion paths/sec	0.8	1.4	1.8
Conv	Million pixels/sec	1250	3500	2.8
FFT	GFLOP/sec	71.4	213	3.0
SAXPY	GBytes/sec	16.8	88.8	5.3
LBM	Million lookups/sec	85	426	5.0
Solv	Frames/sec	103	52	0.5
SpMV	GFLOP/sec	4.9	9.1	1.9
GJK	Frames/sec	67	1020	15.2
Sort	Million elements/sec	250	198	0.8
RC	Frames/sec	5	8.1	1.6
Search	Million queries/sec	50	90	1.8
Hist	Million pixels/sec	1517	2583	1.7
Bilat	Million pixels/sec	83	475	5.7

性能总结

- GPU（480）的内存带宽是Core i7的4.4倍
- GPU 有最高13.1倍的单精度SIMD运算峰值性能
- CPU使用cache，在RC、Sort、Search等程序集上有较明显的效果，对其他程序集也有帮助。
- GPU支持聚集-分散技术
- GPU上同步与存储一致性的缺失，限制了GPU性能

多线程 DGEMM

- Use OpenMP

```
void dgemm (int n, double* A, double* B, double* C)
{
    #pragma omp parallel for
    for ( int sj = 0; sj < n; sj += BLOCKSIZE )
        for ( int si = 0; si < n; si += BLOCKSIZE )
            for ( int sk = 0; sk < n; sk += BLOCKSIZE )
                do_block(n, si, sj, sk, A, B, C);
}
```

谬误与陷阱

- 并行计算机不遵循Amdahl定律
 - 我们能实现线性加速，但只能在弱比例缩放程序上实现
- 峰值性能可代表实际性能
 - 市场营销人员喜欢这种说法
 - Amdahl定律已经指出，单核达到峰值已经很困难，并行机达到完美加速也很困难
- 不开发软件来利用和优化多处理器体系结构
 - 比如：使用单个锁来处理资源
 - 使得请求串行化，即使这些请求能够被并行处理。
 - 使用更小粒度的锁

小结

- 目标：使用更多的处理器，达到更高性能
- 存在的困难：
 - 开发并行软件
 - 设计适当的体系结构
- 软件即服务（SaaS）的重要性日益增加
- 每美元的性能和每焦耳的性能驱动着移动客户端硬件和仓储级计算机（WSC）硬件的发展

第九章 并行处理器

- 并行处理器与并行程序
- GPU与多处理器网络拓扑
- 多处理器测试基准和性能模型